



Higiene de Estado de Sesión

➤ `run_state.json` como vista derivada, guards en los servidores MCP, limpieza determinista y eslabón semántico de bitácoras

Resumen ejecutivo

Problema: `run_state*.json` son *vistas derivadas* de los logs append-only `session_log_*.jsonl` (la fuente de verdad). Una vista puede *sobrevivir a su corrida* y quedar huérfana, contaminando las respuestas de los servidores MCP con el estado de *otra* ejecución. Además, la memoria semántica del proyecto (por QUÉ se decidió, qué se descartó) vive *exclusivamente* en las bitácoras de sesión `Sessions/`, y no había mecanismo determinista para asegurar que existieran ni que tuvieran la estructura mínima.

Solución (aprobada por el usuario): dos pillars de higiene, ambos a **0 créditos** OpenRouter:

- Bajo nivel (estado de ejecución):** script determinista (`execution/estado_sesion.py`) con `check + clean`, una guardia por `mtime` en los 5 servidores MCP, edge case documentado en la directiva, y regla de *state hygiene* en `AGENTS.md`.
- Alto nivel (contexto semántico):** script determinista (`execution/bitacoras.py`) con nueva `-tema` (plantilla canónica que nunca sobrescribe) y `check [-today]` (valida nombre, fecha y secciones mínimas de las bitácoras recientes), edge case de “bitácora no escrita” en la directiva, y regla de cierre en `AGENTS.md`.

1 El problema: vistas que sobreviven al run

1.1 Modelo de estado en ELECTRONICA

El rastreo de flujos multi-paso sigue una separación estricta entre *fuentes de verdad* y *vistas*:

Artefacto	Rol	Naturaleza
<code>session_log_<run>.jsonl</code>	Fuente de verdad	Append-only, inmutable
<code>run_state.json</code>	Vista global del último flujo	Sobrescribible
<code>run_state_<run>.json</code>	Vista de un run concreto	Regenerable

💡 Concepto clave

Concepto clave: un `run_state.json` puede *quedar huérfano*. Si el flujo termina (evento `flujo/fin` en su log) pero la vista no se limpia, el archivo sigue existiendo en `.tmp/` apuntando a un run que ya no está activo. Cualquier consumidor que lo lea *sin verificar* reportará estado de una corrida terminada — o peor, de una corrida distinta.

1.2 Caso real detectado

Durante la auditoría de continuidad se encontró exactamente este caso:

Caso de estudio — vista huérfana de un dry-run

- `.tmp/run_state.json` apuntaba a `flujo-repo-a-skill-2026-09-09T12-29-54` (un dry-run de prueba de trazabilidad).
- Su `log session_log...jsonl` tenía **6 eventos** y terminaba con `flujo/fin`: la corrida *ya había terminado*.
- El estado referenciaba `deepseek/deepseek-v4-pro`, modelo **discontinuado** (sustituido por `deepseek-v4.1-flash`).
- Los servidores MCP leen `run_state.json global` como “estado del flujo recién lanzado” → riesgo real de responder con el contexto de otra corrida.

La trazabilidad append-only estaba íntegra: la cadena de hashes del log era válida (integrity OK). El fallo no era del log, sino de *confiar en la vista sin cruzar contra la fuente*.

2 La solución en tres capas

Siguiendo la arquitectura determinista, el refuerzo cubrió las tres capas:

Capa	Artefacto	Aporte
L3 Ejecución	<code>execution/estado_sesion.py</code>	Diagnóstico y purga de vistas huérfanas (<code>check/clean</code>)
L3 Ejecución	<code>execution/bitacoras.py</code>	Plantilla (nueva) y validación de bitácoras semánticas (<code>check</code>)
L1 Directiva	<code>directives/trazabilidad_sesiones.md</code>	Edge cases “run_state huérfano” y “bitácora sin escribir”
L2 Orquestación	<code>AGENTS.md</code> + guards en MCP	Higiene de bajo nivel al arrancar; cierre de bitácora al finalizar

2.1 Capa de ejecución: `execution/estado_sesion.py`

2.1.1 check — diagnóstico, nunca borra

Escanea `run_state.json` y `run_state_*.json`, cruza cada vista contra su log y emite un veredicto por artefacto:

```
1 $ python3 execution/estado_sesion.py check
2 {
```

```
3  "checks": [ { "archivo": ".../.tmp/run_state.json",
4                "run_id": "flujo-repo-a-skill-...",
5                "veredicto": "huerfano",
6                "razon": "la corrida termino (flujo/fin); la vista
                        sobrevivio.",
7                "modelo_obsoleto": "deepseek-v4-pro" } ],
8  "veredicto_global": "atencion", "anomalias": 1
9 }
```

Veredictos posibles y significado:

Veredicto	Significado
huerfano	El log existe y su último evento es flujo/fin: la vista sobrevivió a la corrida terminada
vigente	Log sin flujo/fin: corrida en curso o reanudable
no_verificable	No hay log (flujo sin trazabilidad): no se borra por falta de certeza
corrupto	JSON inválido o sin run_id (escritura a medias)

Además detecta **modelos obsoletos** por substring (deepseek-v4-pro) y lo reporta como advertencia (no justifica borrado por sí solo).

2.1.2 clean — borra solo con certeza

```
1 $ python3 execution/estado_sesion.py clean          # purga vistas
    huerfanas
2 $ python3 execution/estado_sesion.py clean --dry-run # lista sin
    borrar
3 $ python3 execution/estado_sesion.py check --run <id> # acotar a un run
```

Regla de oro del -clean: **borra únicamente** las vistas cuya corrida terminó con flujo/fin (certeza absoluta). **Nunca** borra los logs append-only (fuente de verdad inmutable), y **no** toca los no_verificable.

Recuerda: check solo diagnostica. El borrado es intencional, vía clean, y está restringido a lo que el criterio anterior puede confirmar. Un flujo sin trazabilidad (no_verificable) se advierte pero jamás se purga automáticamente.

2.2 Capa de directiva: trazabilidad_sesiones.yaml

Se amplió la directiva con dos edge cases específicos (protocolos de recuperación):

```
1 - case: "run_state.json obsoleto o huerfano al inicio de una sesion/
    consulta MCP"
2   recovery: >
```

```

3     Ejecutar execution/estado_sesion.py check y purgar con clean si
        marca
4     huérfano/corrupto. Nunca tocar los logs (fuente inmutable).
5
6 - case: "No se escribió la bitacora semantica de la sesion"
7   recovery: >
8     Al iniciar: si no existe bitacora del dia, crearla con
9     python3 execution/bitacoras.py nueva --tema "<tema>".
10    Al cerrar: rellenarla y validar con bitacoras.py check.
11    El contexto de ALTO nivel (por que se decidio, que se descarto)
        vive
12    EXCLUSIVAMENTE en Sessions/. Sin bitacora, la proxima sesion pierde
13    contexto y volveria a investigar desde cero.

```

bitacoras.py está referenciado en la lista de scripts de la directiva y el edge case cubre la continuidad semántica entre sesiones.

2.3 Capa de orquestación: guards en los MCP y AGENTS.md

2.3.1 Guardia por mtime en los 5 servidores MCP

Los servidores mcp_evaluar, mcp_diagnostico, mcp_analizar, mcp_elaborar y mcp_docs siguen el mismo patrón: lanzan el flujo con subprocess.run y, si termina OK, leen run_state.json para dar una respuesta rica. La corrección captura el mtime *antes* de lanzar y solo lee la vista si el archivo fue **reescrito durante esta corrida**:

```

1  state_file = os.path.join(project_root, ".tmp", "run_state.json")
2  mtime_before = (os.path.getmtime(state_file)
3                  if os.path.exists(state_file) else None)
4  try:
5      resultado = subprocess.run(cmd, capture_output=True, text=True,
6                                 encoding="utf-8", cwd=project_root)
7      state_data = {}
8      if os.path.exists(state_file):
9          try:
10             if mtime_before is not None and \
11                os.path.getmtime(state_file) <= mtime_before:
12                 pass # vista de OTRA corrida: no reportar
13             else:
14                 with open(state_file, "r", encoding="utf-8") as f:
15                     state_data = json.load(f)
16         except Exception:
17             state_data = {}

```

Recuerda

⚠ Corrección de diseño importante: la primera versión del guard comprobaba "flujo/fin" en el log para descartar la vista. **Ese criterio era inválido:** un flujo que corre *bien* también emite flujo/fin, así que se habría descartado el estado legítimo de la corrida que acaba de terminar con éxito. El criterio correcto para el MCP es temporal: "¿el run_state.json fue reescrito *durante* este lanzamiento?" → comparación de mtime. Se corrigió en los 5 servers antes del commit.

2.3.2 Regla de arranque y cierre en AGENTS.md

```

1 - **State hygiene (freshness):** run_state*.json are *derived views* of
   the
2   append-only session_log*.jsonl; a view can outlive its run (orphan)
   and poison
3   MCP responses. At the start of each session, run 'python3 execution/
   estado_sesion.py
4   check'; purge confirmed orphans with '... clean' (never touches the
   immutable logs).
5
6 - **Session logs (continuity):** ...
7   - At the start: create bitácora via 'python3 execution/bitacoras.py
   nueva --tema "<tema>"
8   - At session close: run '... check' and fix any missing day-log or
   empty sections

```

Concepto clave

💡 Dos niveles de memoria, dos instrumentos: el *bajo nivel* (datos reproducibles: log + run_state) se gestiona con estado_sesion.py. El *alto nivel* (semántica: por qué se decidió, qué se descartó) se gestiona con bitacoras.py. Sin ninguno de los dos, la próxima sesión re-investiga desde cero (tokens/créditos quemados).

3 Eslabón semántico: bitácoras de sesión**Concepto clave**

💡 El problema: la salud de ejecución (sección anterior) blinda el *bajo nivel*: datos reproducibles, deterministas, verificables. Pero el *contexto de alto nivel* — por qué se decidió algo, qué se descartó, las intenciones del usuario, los matices que no caben en un log JSON — vive **exclusivamente** en las bitácoras markdown de Sessions/. Si no se escriben, la siguiente sesión *no puede recuperar* ese razonamiento y volvería a investigar desde cero, quemando tokens y tomando decisiones sin contexto.

3.1 execution/bitacoras.py

Script determinista (0 créditos) con dos subcomandos:

3.1.1 nueva -tema <tema>

Genera la plantilla canónica de la sesión de HOY (nunca sobrescribe un archivo existente). Formato del nombre: YYYY-MM-DD_<tema>.md (slug ASCII con transliteración de acentos). Plantilla:

```

1 # 2026-09-11      Tema de la sesión
2
3 ## Tema
4 (1 frase sobre qué se abordó)
5
6 ## Contexto
7 (Por qué se aborda, eventos detectados)
8
9 ## Decisiones (usuario)
10 (Acuerdos explícitos del usuario      NO inventar)
11
12 ## Actividades
13 (Pasos, scripts, salidas, veredictos)
14
15 ## Pendientes
16 (Queda abierto, pendientes acordados de otras sesiones)

```

3.1.2 check [-today]

Diagnostica la estructura de las bitácoras existentes: valida nombre (YYYY-MM-DD_<tema>.md), fecha parseable (sin futuros) y las cuatro secciones mínimas. Clasifica cada bitácora en tres periodos:

Periodo	Significado
hoy	Sesión del día en curso — debe existir y cumplir plantilla
reciente	Desde la adopción de la plantilla — debe cumplir estructura
legado	Antes de la plantilla — formato heredado, informativo, no penaliza

Solo las anomalías en hoy/reciente cuentan para veredicto_global y el exit code. El check también reporta si la sesión del día ya dejó bitácora (sesion_hoy.hay_bitacora).

```

1 $ python3 execution/bitacoras.py check
2 {
3     "bitacoras": [...],
4     "sesion_hoy": {"fecha": "2026-09-11",
5                     "hay_bitacora": true,
6                     "archivos": ["Sessions/2026-09-11_....md"]},
7     "anomalias": 0, "accionables": 0,
8     "veredicto_global": "ok"
9 }

```

3.2 Hábito codificado en AGENTS.md

La regla en AGENTS.md vincula el uso del script al flujo de trabajo:

- **Al inicio:** buscar bitácora previa del tema → crear la nueva con `bitacoras.py nueva -tema`.
- **Al cerrar:** rellenar las secciones y ejecutar `bitacoras.py check`; si reporta anomalía, corregir antes de terminar.

Recuerda

Las bitácoras legado no se reescriben: las sesiones anteriores a la adopción de la plantilla (2026-09-11) usaban otro formato. Se listan informativas para mantener trazabilidad histórica; solo las nuevas (desde hoy) deben seguir la plantilla canónica.

4 Verificación (0 créditos)

4.1 Guardia MCP: dos casos controlados

Se replicó la lógica exacta del server contra dos escenarios:

Caso A — vista huérfana (no reescrita)

```
1 run_state.json escrito hace 1h (mtime viejo).
2 mtime_before = getmtime() # la corrida NO reescribe el archivo
3 resultado: state_data == {} # << no se reporta estado ajeno
```

Resultado: **se descarta** la vista de otra corrida. ✓

Caso B — estado reescrito por la corrida

```
1 mtime_before = getmtime()
2 el flujo reescribe run_state.json # mtime > mtime_before
3 resultado: state_data == {"run_id": "...", "context": {...}} # <<
  se usa
```

Resultado: **se conserva** el estado legítimo. ✓

4.2 Limpieza real del caso detectado

```

1 $ python3 execution/estado_sesion.py check
2 # -> veredicto "huerfano" + advertencia deepseek-v4-pro
3 $ python3 execution/estado_sesion.py clean
4 # -> eliminado: .tmp/run_state.json
5 $ python3 execution/estado_sesion.py check
6 # -> checks: [] veredicto_global: ok (log immutable intacto)

```

El log append-only session_log_flujo-repo-a-skill-2026-09-09T12-29-54.jsonl permaneció **intacto** (fuente de verdad intacta).

5 Alineación con la arquitectura

- **Determinismo:** mismo estado de .tmp/ → mismo veredicto. El script no razona ni consume créditos.
- **Separación de responsabilidades:** la decisión de *cuándo* una vista es obsoleta o una bitácora está incompleta vive en código (capas 1 y 3), no en el chat del orquestador.
- **Autocuración:** la guardia de mtime evita que un MCP “recupere” contexto basura tras un flujo que no llegó a escribir estado; el bitacoras.py check evita cerrar una sesión sin dejar rastro semántico.
- **Retry budget respetado:** sin reintentos infinitos; el criterio es binario y verificable.

Ejemplo

</> Ejemplo de uso cotidiano: tras el reinicio de una sesión (o antes de reanudar un flujo multi-paso), basta con

```
python3 execution/estado_sesion.py check
```

para saber si run_state.json es fiable. Si reporta huerfano, un clean lo purga en un paso.

6 Pendientes y líneas siguientes

- Considerar que los flujos emitan su run_id al final del stdout (permitiría validación robusta sin depender de mtime).
- Extender check para limpiar run_state_*.json *por run* de forma aún más agresiva solo si el usuario lo solicita.
- Integrar bitacoras.py check como recordatorio automático de cierre de sesión (en flujo de arranque o directiva).

- El pendiente de docs/AGENTE_IA/orquestador_repo_a_skill.mp4 permanece en manos del usuario (fuera del alcance de esta sesión).